ZipMould

Gabriel Mitelman Tkacz · Maio de 2026

Programa de Pós-Graduação em Engenharia Elétrica e Computação

Sumário

1. Slime mould algorithm: A new method for stochastic optimization, Li et al. (~7 min)

O que é o Slime Mould Algorithm (SMA)?

2. ZipMould (~7 min)

Como adaptar o SMA contínua a um problema combinatório discreto

3. Demonstração (~5 min)

Reprodução ao vivo no visualizador

O que é o *Physarum polycephalum*?

- O **fungo mucilaginoso acelular** é um único organismo multinucleado, sem sistema nervoso.
- Procura alimento estendendo uma **rede venosa** pelo substrato.
- ***Feedback* positivo**: mais alimento → fluxo citoplasmático mais rápido → veias mais grossas.
- ***Feedback* negativo**: ramos sem alimento retraem.
- Já foi usado para resolver labirintos, replicar a **rede ferroviária de Tóquio** e agir como um otimizador distribuído sem controle central.

> O algoritmo bioinspirado transforma a metáfora em três regras de atualização: ***approaching food***, ***wrapping food*** (*feedback* na espessura das veias) e ***grabbling food*** (amplitude do bio-oscilador).

Li, Chen, Wang, Heidari & Mirjalili (2020), Future Generation Computer Systems 111, 300–323. Tero et al. (2010), Science 327, 439–442.

SMA Eq. (2.1) — *aproximar-se do alimento*

$$\vec{X}(t+1) = \begin{cases} \vec{X}_b(t) + \vec{v}_b \cdot \left(\vec{W} \cdot \vec{X}_A(t) - \vec{X}_B(t)\right), & r < p \\ \vec{v}_c \cdot \vec{X}(t), & r \geq p \end{cases}$$

- $\vec{X}_b$ — melhor indivíduo encontrado até agora.
- $\vec{X}_A, \vec{X}_B$ — dois indivíduos aleatórios que definem uma direção de exploração.
- $\vec{W}$ — **peso** derivado do *ranking* de *fitness* (Eq. 2.5, próximo *slide*).
- $\vec{v}_b \in [-a, a]$, em que $a = \operatorname{arctanh}(\frac{1-t}{T})$.
- $\vec{v}_c$ diminui linearmente de 1 para 0.
- $p = \tanh|S(i) - DF|$ — limiar adaptativo para alternar o comportamento.

Li et al. (2020), §2.3.1.

SMA Eq. (2.5) — o peso $\vec{W}$

$$W_i = \begin{cases} 1 + r \cdot \log\left(\dfrac{bF - S(i)}{bF - wF} + 1\right), & i \in \text{metade superior (avaliação boa)} \\ 1 - r \cdot \log\left(\dfrac{bF - S(i)}{bF - wF} + 1\right), & i \in \text{metade inferior (avaliação ruim)} \end{cases}$$

- A **metade superior** (UB) da população puxa para áreas favoráveis → ***feedback* positivo**.
- A **metade inferior** (LB) é empurrada para longe → ***feedback* negativo** simulando a retração de veias sem alimento.
- O $\log$ suaviza a taxa de mudança; $r \sim \mathcal{U}(0, 1)$ mantém a resposta estocástica.
- Captura a "preferência" do fungo mucilaginoso pelo ***ranking* de *fitness*** e não pelo *fitness* em si.

Li et al. (2020), §2.3.2 — modelo matemático de "Wrap food".

SMA Eq. (2.7) — a atualização completa

$$\vec{X}^* = \begin{cases} \text{rand} \cdot (UB - LB) + LB, & \text{rand} < z \\ \vec{X}_b(t) + \vec{v}_b \cdot (W \cdot \vec{X}_A - \vec{X}_B), & r < p \\ \vec{v}_c \cdot \vec{X}(t), & r \geq p \end{cases}$$

- Três casos, **por indivíduo, por iteração**:
  i. ***z-branch*** (prob. $z = 0.03$): reinício aleatório no espaço de busca.
  ii. Caso ***approach*** (prob. $\approx p$): explora a melhor solução com perturbação ponderada por W.
  iii. Caso ***oscillate***: contrai a solução em direção à origem com v_c.

> A *z-branch* permite que o SMA escape de ótimos locais sem influência externa explícita e sem reiniciar a população inteira.

Li et al. (2020), §2.3.2 Eq. (2.7); z escolhido como 0.03 a partir do sweep de sensibilidade §3.4.

A evolução de v_b e v_c

v_b: amplitude saturante

$$v_b \in [-a, a], \quad a = \operatorname{arctanh}(1 - \frac{t}{T})$$

No início: $a \to \infty$, causando saltos grandes. No fim: $a \to 0$ resultando em exploração local mais fina.

v_c: contração linear

$$v_c \in [-1, 1], \quad v_c \to 0 \text{ quando } t \to T$$

Amortece o caso *oscillate*. No fim da execução, $v_c \cdot X \approx 0$: o agente praticamente para de se mover sozinho.

Efeito combinado

Iterações iniciais: ***exploration*** domina porque v_b é grande.

Iterações finais: ***exploitation*** domina quando v_b e v_c diminuem.

Da metáfora onde o fungo decide se aproxima da fonte atual ou procura outra, aqui, isso aparece como amplitude de oscilação.

Li et al. (2020), §2.3.3 "Grabble food", Figura 5.

O algoritmo SMA

```
INITIALISE population X_1 ... X_n at random in [LB, UB]

FOR t = 1 ... T:
    evaluate fitness S(i) for all i
    sort population, identify bF, wF, X_b
    compute W via Eq. (2.5)                    # pesos positivos/negativos por ordenação
    FOR each individual i:
        sample r ~ U(0,1), rand ~ U(0,1)
        update v_b, v_c, p                     # evolução
        IF rand < z:
            X_i ← random restart in [LB, UB]
        ELIF r < p:
            X_i ← X_b + v_b * (W * X_A - X_B)  # aproximação
        ELSE:
            X_i ← v_c * X_i                    # oscilação

RETURN bF, X_b
```

- Um *loop* externo, três casos internos, sem derivadas e sem gradientes.
- Três hiperparâmetros: população n, iterações T, probabilidade de reinício z. Os valores de v_b, v_c são derivados, não ajustados.

O *puzzle* Zip

Definição formal

Dada uma grade $G_{N \times N}$ com:

- K *waypoints* ordenados $w_1, w_2, \ldots, w_K$
- Um conjunto de **paredes** (arestas proibidas entre células adjacentes)
- Um conjunto de células **bloqueadas**

Queremos encontrar um **caminho Hamiltoniano** $\pi_1, \ldots, \pi_L$ tal que:

1. células consecutivas sejam 4-adjacentes e não separadas por parede,
2. *waypoints* apareçam em **ordem crescente**,
3. $\pi_1 = w_1$ e $\pi_L = w_K$.

> O problema de decisão é **NP-completo**, mas é resolvível em prática para $N \leq 10$, uma faixa onde metaheurísticas valem a pena.

Por que não dá para aplicar o SMA contínuo diretamente

O SMA de Li existe em $\mathbb{R}^d$

- $\vec{X}_A - \vec{X}_B$ é um vetor de direção euclidiano.
- $v_b \cdot W$ escala uma amplitude em espaço contínuo.
- O passo é apenas soma vetorial.

Zip é um problema em grafo

- "Posição" é um caminho Hamiltoniano parcial, não uma coordenada.
- $\vec{X}_A - \vec{X}_B$ é indefinido entre dois caminhos.
- A conclusão natural é o uso de arestas, não coordenadas de pontos.

A ponte

A saída é pela estigmergia, como no **Ant Colony Optimisation** (Dorigo, 1992): o feromônio τ nas arestas passa a ser o estado do agente.

Em seguida, levamos a **dinâmica de atualização** do SMA (evolução de v_b e v_c, pesos assinados por *ranking* e *z-restart*) para o feromônio, não para uma coordenada.

O pipeline ZipMould

```
Entrada (grade + K pontos) → Checagem prévia O(N²) → Código Python popualação x T → Avaliação + registro
```

Checagens de viabilidade

- *Waypoint* alcançável e não bloqueado
- Subgrafo livre **conectado** (BFS a partir de w_1 cobre todas as células livres)
- **Limite de paridade**: $|F_0 - F_1| \leq 1$ na coloração de tabuleiro
- **Paridade dos extremos** consistente com w_1, w_K

Se qualquer um desses falha, o *puzzle* não é resolvível e o *kernel* nem roda.

Etapa 1: construção estilo ACO

Cada *walker* escolhe um vizinho 4-adjacente via ***softmax*(feromônio + heurística)**:

$$P(c \to c') \propto \exp\Big(\alpha \cdot \tau_{cc'} + \beta \cdot \log \eta_{c'}\Big)$$

Heurística combinada:

$$\eta_{c'} = \mathrm{softplus}(h_m)^{\gamma_m} \cdot \mathrm{softplus}(h_w)^{\gamma_w} \cdot \mathrm{softplus}(h_a)^{\gamma_a} \cdot \mathrm{softplus}(h_p)^{\gamma_p}$$

Heurística	Função
h_m (Manhattan)	Puxar para o próximo *waypoint*
h_w (Warnsdorff)	Preferir baixo grau; consumir becos sem saída cedo
h_a (Articulação)	Rejeitar movimentos que **desconectam** o subgrafo livre não visitado
h_p (Paridade)	Manter $\lvert F_0 - F_1 \rvert \leq 1$ após o movimento

A combinação por softplus aceita sinais mistos; defaults α = 1, β = 2 seguem ACO (Dorigo & Stützle, 2004).

Etapa 2: atualização do feromônio no estilo SMA

```
progress = float(t) / float(T)
v_b = math.tanh(1.0 - progress)        # inspirado em Li, LIMITADO (cf. arctanh)
v_c = 1.0 - progress                   # Li 2.4 literal

# Pesos assinados por ordenação ∈ [-1, +1]: melhor agente = +1, pior = -1, mediana ≈ 0
denom = float(n - 1)
weights[i] = (float(n) - 2.0 * float(r) + 1.0) / denom

# Atualização por aresta, análogo da Eq. (2.7) de Li
new_val = v_c * tau[s, e] + v_b * deposit[s, e]

# z-branch ANÁLOGA (não literal): aplicada ao FEROMÔNIO,
# não à posição do agente como em Li
if z > 0.0 and np.random.random() < z:
    new_val = np.random.normal(0.0, tau_max / 4.0)
```

> O peso **assinado** por *ranking* é o análogo discreto do W de Li: *walkers* da metade superior *depositam* feromônio; os da metade inferior o *evaporam* nas mesmas arestas.

O que mudou em relação a Li

SMA (contínuo)	ZipMould (discreto)	Por que a mudança
Estado $\vec{X} \in \mathbb{R}^d$	Feromônio $\tau \in \mathbb{R}^m$ (m = #arestas)	Não há espaço de coordenadas; arestas carregam memória
$W_i = 1 \pm r \log(\cdot)$	$W_i = \frac{n-2r+1}{n-1}$	*Ranking* linear é limitado, sem singularidade de $\log$
$v_b \sim \mathcal{U}(-a, a)$ **por agente** ($a = \operatorname{arctanh}(\frac{1-t}{T})$): oscilador **estocástico**, ilimitado em $t = 0$	$v_b = \tanh(1 - \frac{t}{T})$: escalar **determinístico** por iteração, em $[0, \tanh 1]$	Saturação estabiliza depósitos discretos; a perda da oscilação estocástica de v_b é compensada pela amostragem *softmax* dos *walkers*
Atualização com *switching* em três casos (z / *approach* / *oscillate*)	Soma **única** $v_c\tau + v_b\Delta$ + ruído *z-branch*	Todos os ingredientes de Li em cada passo; sem sorteio de *branch* por indivíduo
Direção aleatória $X_A - X_B$	Substituída por **agregado ponderado por *ranking*** $\sum_w W_w \cdot 1[\text{agente } w \text{ usou aresta } e]$	"Diferença de dois pontos aleatórios" é indefinida em grafo

Matriz de ablação

Duas escolhas de design

- **Modo de feromônio**:
  - *unified* tem um τ por aresta no caminho inteiro
  - *stratified* tem um τ por par (aresta, segmento entre *waypoints*)
- **Sinal**:
  - *signed* atrai e repele (análogo completo do SMA)
  - *positive* apenas a metade superior deposita (mais próximo de ACO)

4 condições x 4 *baselines*

	unified	*stratified*
signed	A	B
positive	C	D

Hipóteses pré-registradas

1. ***signed*** > ***positive*** em *puzzles* difíceis (*feedback* negativo quebra simetria)
2. ***stratified*** > ***unified*** quando K é grande (memória por segmento importa)
3. ZipMould > *baseline* ACO puro no *split* de teste *held-out*

Teste pareado de McNemar com correção FDR nas 4 condições x 4 baselines x seeds.

Baselines + protocolo estatístico

Baseline	Feromônio	Depósito	Observações
aco-vanilla	*unsigned, unified*	$\propto$ *fitness*	Evaporação ρ clássica, sem ruído de *restart*
heuristic-only	—	—	Guloso apenas em η, mede a força das heurísticas
random-walk	*uniform*	—	Piso de exploração pura
backtracking	—	—	DFS exaustivo com poda por paridade + *articulation*

Protocolo pré-registrado

- *Splits* ***train***, ***dev***, ***test*** estratificados pela dificuldade do *puzzle*.
- ***Held-out test set***: resultados computados uma vez.
- **Teste pareado de McNemar** sobre sucesso ou fracasso em cada *puzzle*.
- **Correção FDR Benjamini-Hochberg** na matriz 4×4 de condições.
- Todas as *seeds* são reproduzíveis.

Demonstração ao vivo

Indo para o visualizador...

https://app.zipmould.tkacz.dev.br/

Referências

- **Li, S., Chen, H., Wang, M., Heidari, A. A. & Mirjalili, S.** (2020). *Slime mould algorithm: A new method for stochastic optimization*. *Future Generation Computer Systems* 111, 300–323.
- **Dorigo, M. & Stützle, T.** (2004). *Ant Colony Optimization*. MIT Press.
- **Mirjalili, S. & Lewis, A.** (2016). *The Whale Optimization Algorithm*. *Advances in Engineering Software* 95, 51–67.
- **Tero, A. et al.** (2010). *Rules for Biologically Inspired Adaptive Network Design*. *Science* 327, 439–442.
- **Warnsdorff, H. C.** (1823). *Des Rösselsprungs einfachste und allgemeinste Lösung*.
- **McNemar, Q.** (1947). *Note on the sampling error of the difference between correlated proportions or percentages*. *Psychometrika* 12, 153–157.